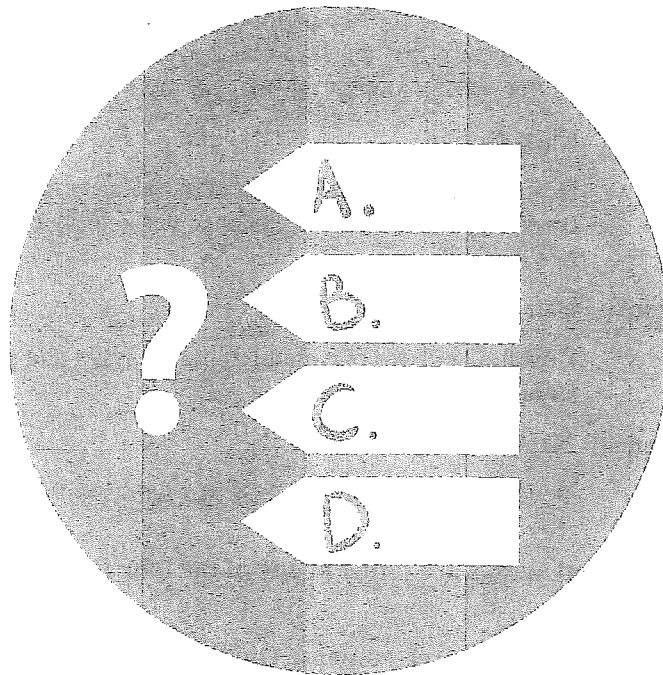


Practice Exams



Notes:

A blank multiple-choice answer form is available at

www.skylit.com/beprepared/MCQAnswerForm.pdf

Remember the topics of the free-response questions in each exam:

Question 1: Methods and Control Structures

Develop code for two methods or one method and one constructor of a partially written Java class. Use `if-else` statements and `for` and/or `while` loops; call `String` methods.

Question 2: Class Design

Design and implement a Java class from scratch. Solution should consist of a class header, instance variable(s), one constructor, and one method.

Question 3: Data Analysis with `ArrayList`

Develop code for one method that analyzes and processes data stored in an `ArrayList`.

Question 4: 2D Array

Develop code for one method that works with a 2D array.

No one-dimensional arrays are tested in any of the four free-response questions.

`ArrayList` is tested only in free-response Question 3.

From the College Board's notes:

- Assume that the classes listed in the Java Quick Reference have been imported where appropriate.
- Unless otherwise noted in the question, assume that parameters in method calls are not null and that methods are called only when their preconditions are satisfied.
- In writing solutions for each question, you may use any of the accessible methods that are listed in classes defined in that question. **Writing significant amounts of code that can be replaced by a call to one of these methods will not receive full credit.**

Practice Exam #1

SECTION I

Time — 90 minutes
Number of questions — 42
Percent of total grade — 55

1. Assume that the boolean variables `x`, `y`, and `z` have been properly declared and initialized. Given that `x` is true, `y` is true, and `z` is false, which of the following expressions will evaluate to false?

- (A) `(x && y) || z`
- (B) `(x || y) && z`
- (C) `y || (x && z)`
- (D) `x && (y || z)`

2. Given the declarations

```
int p = 5;  
int q = 3;
```

which of the following expressions evaluate to 7.5?

- (A) `(double) (p * q / 2);`
- (B) `(double)p * (double) (q / 2);`
- (C) `(double)p * (double)q / 2;`
- (D) None of the above

3. What are the smallest and the largest possible values of `x` after the following statement has been executed?

```
int x = (int) (Math.sqrt(4*Math.random()) + 0.5);
```

- (A) 0 and 1
- (B) 0 and 2
- (C) 1 and 2
- (D) 1 and 3

4. Consider the following code segment.

```
ArrayList<Integer> list = new ArrayList<Integer>();

for (int i = 1; i <= 8; i++)
{
    list.add(i);
}

for (int i = 0; i < list.size(); i++)
{
    list.remove(i);
}

for (Integer x : list)
{
    System.out.print(x + " ");
}
```

What is printed when the code segment is executed?

- (A) IndexOutOfBoundsException
 - (B) 1 3 5 7
 - (C) 2 4 6 8
 - (D) No output, because the resulting list is empty
5. Consider the following code segment.

```
int[] factors = {2, 3, 4, 7, 2, 5};
int product = 1;

for (int i = 1; i < factors.length; i += 2)
{
    product *= (factors[i] % factors[i-1]);
}
```

What is the value of product after the code segment is executed?

- (A) 1
- (B) 2
- (C) 3
- (D) 5

6. Consider the following statements.

```
String str = "\\*\\n*\\\";
System.out.println(str + " " + str.length());
```

What is printed when the statements are executed?

- (A) `\\*\\n*\\ 5`
 - (B) `\\*\\n*\\ 8`
 - (C) `\\*`
`*\\ 5`
 - (D) `\\*`
`*\\ 7`
7. Consider the following code segment. It is supposed to calculate and print the sum $1 + 2 + \dots + 20$.

```
int count = 0;
int sum = 0;
while (count < 20)
{
    sum += count;
}
System.out.println(sum);
```

Which of the following statements best describes the result?

- (A) The printed result will be incorrect, smaller than expected by 20.
 - (B) 0 is printed
 - (C) 20 is printed
 - (D) Nothing is printed, because the program goes into an infinite loop.
8. The statement

```
double x = samples[ < missing index > ];
```

assigns to `x` an element of the array `samples`, randomly chosen with equal probabilities. Which of the following can replace `< missing index >`?

- (A) `(int) (samples.length * Math.random())`
- (B) `(int) (samples.length * Math.random()) - 1`
- (C) `(int) (samples.length * Math.random()) + 1`
- (D) `(int) (samples.length * Math.random() + 0.5)`

9. Consider the following code segment.

```
Integer n = 3;           // Line 1
Double x = 0.14;         // Line 2
System.out.println(n + x);
```

What is the result when the code segment is compiled/executed?

- (A) 3 is printed
 - (B) 3.14 is printed
 - (C) Syntax error on Line 1
 - (D) Syntax error on Line 2
10. Assume that `int` variables `x` and `y` have been properly declared and initialized. The expression

```
!(x > y && y <= 0)
```

is equivalent to which of the following?

- (A) `x <= y || y > 0`
 - (B) `x > y && y <= 0`
 - (C) `x > y || y < 0`
 - (D) `x <= y && y <= 0`
11. Consider the following class.

```
public class Sphere
{
    public static final double PI = 3.14159;

    public static double volume(int r)
    {
        return 4 / 3 * PI * Math.pow(r, 3);
    }
}
```

Which of the following statements about this code is true?

- (A) The class will not compile because no constructors are defined.
- (B) The class will not compile because the `volume` method is declared `static`.
- (C) `Math.pow(r, 3)` cannot be used because `r` is an `int`.
- (D) The class compiles with no errors but the `volume` method returns a smaller value than the expected $\frac{4}{3}\pi r^3$.

12. Consider the following method.

```
public double goFigure(int n)
{
    n %= 7;
    return (double)(12 / n);
}
```

What is printed when the following code segment is executed?

```
int n = 12;
System.out.print(goFigure(n));
System.out.println(" " + n);
```

- (A) 2.0 12
- (B) 2.4 12
- (C) 2.0 5
- (D) 2.4 5

13. The method

```
public void addStars(ArrayList<String> words)
```

is intended to append "*" at the end of each string in `words` without changing the order of elements in `words`. Which of the following implementations of `addStars` will work?

- (A)

```
for (String word : words)
{
    word += "*";
}
```
- (B)

```
for (int k = 0; k < words.size(); k++)
{
    words.set(k, words.get(k) + "*");
}
```
- (C)

```
for (int k = 0; k < words.size(); k++)
{
    String word = words.remove(k) + "*";
    words.add(k+1, word);
}
```
- (D) All of the above

14. Consider the following method.

```
public int mysteryCount(int[] p)
{
    int count = 0;
    for (int i = 0; i < p.length; i++)
    {
        count++;
        int j = p[i];
        while (j != i)
        {
            j = p[j];
            count++;
        }
    }
    return count;
}
```

Given

```
int[] arr = {0, 2, 3, 1};
```

what will `mysteryCount(arr)` return?

- (A) 2
- (B) 5
- (C) 10
- (D) 13

15. Which of the following expressions evaluate to true?

- (A) `"Tick-Tock".compareTo("Tick" + "-" + "Tock") == 0;`
- (B) `"Tick-Tock".compareTo("Tock-Tick") < 0;`
- (C) `"Tick-Tock".indexOf("T") > "Tick-Tock".indexOf("t");`
- (D) All of the above

16. Consider the following code segment.

```
ArrayList<String> words = new ArrayList<String>();
words.set(0, "ANT");
```

What is the result when the code segment is executed?

- (A) `NullPointerException`
- (B) `IndexOutOfBoundsException`
- (C) The size of `words` remains 0 and nothing is added to `words`
- (D) The size of `words` becomes 10 and the element at index 0 becomes "ANT"

17. Consider the following code segment with a missing for loop.

```
ArrayList<String> letters = new ArrayList<String>();

letters.add("A");
letters.add("B");
letters.add("C");

< missing "for" loop >

System.out.println(letters);
```

The code segment, when executed, prints

[A!, B!, C!]

Which of the following can replace < missing "for" loop >?

- (A)
- ```
for (String s : letters)
{
 s = s + "!";
}
```
- (B)
- ```
for (String s : letters)
{
    s += "!";
}
```
- (C)
- ```
for (int i = 0; i < letters.size(); i++)
{
 String s = letters.get(i);
 s = s + "!";
}
```
- (D)
- ```
for (int i = 0; i < letters.size(); i++)
{
    letters.set(i, letters.get(i) + "!");
}
```

18. What is printed when the following code segment is executed?

```
int sum = 0, d = -1;

for (int count = 10; count > 0; count--)
{
    sum += d;

    if (d > 0)
    {
        d++;
    }
    else
    {
        d--;
    }
    d = -d;
}

System.out.println(sum);
```

- (A) 5
- (B) -5
- (C) 10
- (D) -10

19. Consider the following class.

```
public class BuddyList
{
    /** Contains the names of buddies. */
    private ArrayList<String> buddies;

    public ArrayList<String> getBuddies()
    {
        return buddies;
    }

    < other instance variables, constructors, and
    methods not shown >
}
```

If `BuddyList myFriends` is declared and initialized in some other class, a client of `BuddyList`, which of the following correctly assigns to `name` the name of the first buddy in the `myFriends` list?

- (A) `String name = myFriends.get(0);`
- (B) `String name = myFriends.buddies[0];`
- (C) `String name = myFriends.buddies.get(0);`
- (D) `String name = myFriends.getBuddies().get(0);`

20. Consider the following code segment.

```
int i = 2;
for (int k = 0; k <= 12; k += i)
{
    System.out.print(k + " ");
    i++;
}
```

What is printed when the statement is executed?

- (A) 0 3 7
- (B) 0 3 7 12
- (C) 2 5 8 11
- (D) 0 3 6 9 12

21. The method `getPrimeFactors` below returns a list of all prime factors of n .

```
public ArrayList<Integer> getPrimeFactors(int n)
{
    ArrayList<Integer> factors = new ArrayList<Integer>();

    for (int p = 2; p <= n; p++)
    {
        while (n % p == 0)
        {
            factors.add(p);
            n /= p;
        }
    }
    return factors;
}
```

How many times is the modulo division operator `%` invoked when `getPrimeFactors(60)` is called?

- (A) 3
- (B) 7
- (C) 8
- (D) 15

22. Consider the following method.

```
public void change(double[] nums, int n)
{
    for (int k = 0; k < n; k++)
    {
        nums[k] = 5.4;
    }
    n = 2;
}
```

What will be stored in `samples` and `len` after the following statements are executed?

```
double[] samples = {1.0, 2.1, 3.2, 4.3};
int len = samples.length;
change(samples, len);
```

- (A) `samples` contains 5.4, 5.4, 5.4, 5.4; `len` is 4
- (B) `samples` contains 5.4, 5.4, 5.4, 5.4; `len` is 2
- (C) `samples` contains 1.0, 2.1, 3.2, 4.3; `len` is 4
- (D) `samples` contains 5.4, 5.4; `len` is 2

23. Consider the following method.

```
public String filter(String str, String pattern)
{
    int pos = str.indexOf(pattern);

    if (pos == -1)
    {
        return str;
    }
    else
    {
        return filter(str.substring(0, pos) +
                      str.substring(pos + pattern.length()), pattern);
    }
}
```

What is printed when the following statement is executed?

```
System.out.println(filter("papaya", "pa"));
```

- (A) `pa`
- (B) `ya`
- (C) `aya`
- (D) `paya`

24. Consider the following incomplete method `cutToAverage`.

```
public double cutToAverage(double[] amps)
{
    double avg = 0.0;

    < missing code >

    return avg;
}
```

It first finds the average `avg` of the values in an array, then replaces every element that exceeds `avg` with `avg`. `cutToAverage` returns `avg`. Which of the following versions of `< missing code >` will fail to work as intended?

- (A)
- ```
for (int k = 0; k < amps.length; k++)
{
 avg += amps[k];
}
avg /= amps.length;
for (int k = 0; k < amps.length; k++)
{
 if (amps[k] > avg)
 amps[k] = avg;
}
```
- (B)
- ```
for (double x : amps)
{
    avg += x;
}
avg /= amps.length;
for (int k = amps.length - 1; k >= 0; k--)
{
    if (amps[k] > avg)
        amps[k] = avg;
}
```
- (C)
- ```
for (double x : amps)
{
 avg += x;
}
avg /= amps.length;
for (double x : amps)
{
 if (x > avg)
 x = avg;
}
```
- (D) All of the above will work.

25. Consider the following class.

```
public class ArrayProcessor
{
 public static void run(int[] arr)
 {
 for (int i = 0; i < arr.length; i++)
 {
 for (int j = arr.length - 1; j > i; j--)
 {
 if (arr[j] < arr[i])
 {
 swap(arr, i, j);
 }
 }
 }
 }

 private static void swap(int[] arr, int i, int j)
 {
 int temp = arr[i];
 arr[i] = arr[j];
 arr[j] = temp;
 }
}
```

How many times will ArrayProcessor's swap method be called when the following code segment is executed?

```
int[] counts = {1, 2, 3, 4, 5, 0};
ArrayProcessor.run(counts);
```

- (A) 5
  - (B) 10
  - (C) 15
  - (D) 30
26. The class `PlayList` provides methods that allow you to represent and manipulate a list of songs, but you are not concerned with how these operations work or how the list is stored in memory. You only know how to initialize and use `PlayList` objects and have no direct access to the implementation of the `PlayList` class or its instance variables. This is an example of:
- (A) method overriding
  - (B) encapsulation
  - (C) method overloading
  - (D) AI use

27. What is the outcome when the following code is compiled/executed?

```
ArrayList<Integer> numbers = new ArrayList<Integer>();
numbers.add(1);
numbers.add(2);
numbers.add(3);
numbers.add(4);

for (int x : numbers)
{
 if (x % 2 == 0)
 {
 numbers.add(5);
 }
}
System.out.println(numbers);
```

- (A) Syntax error
- (B) IndexOutOfBoundsException
- (C) ConcurrentModificationException
- (D) [1, 2, 3, 4, 5, 5] is printed

28. Given the 2D array

```
int[][] magic = {{4, 9, 2},
 {3, 5, 7},
 {8, 1, 6}};
```

what is the value of sum after the following code segment is executed?

```
int sum = 0;

for (int r = 0; r < magic.length; r++)
{
 for (int c = r; c < magic[r].length; c++)
 {
 sum += magic[r][c];
 }
}
```

- (A) 30
- (B) 33
- (C) 35
- (D) 45

29. Consider the following class.

```
public class BankAccount
{
 private int balance;

 public BankAccount() { balance = 0; }
 public BankAccount(int amt) { balance = amt; }

 public int getBalance() { return balance; }
 public void makeDeposit(int amt) { balance += amt; }
}
```

What is printed when the following code segment in a client class is executed?

```
ArrayList<BankAccount> bank = new ArrayList<BankAccount>();
bank.add(new BankAccount());
bank.add(new BankAccount(5));
bank.add(new BankAccount(10));
bank.add(new BankAccount(15));

for (BankAccount customer : bank)
{
 customer.makeDeposit(10);
}
int total = 0;
for (BankAccount customer : bank)
{
 total += customer.getBalance();
}
System.out.println(total);
```

- (A) 0
- (B) 30
- (C) 60
- (D) 70

30. Which of the following statements DOES NOT print 1234?

- (A) `System.out.print(12 * 100 + 34);`
- (B) `System.out.print("12" + 34);`
- (C) `System.out.print(1 + 2 + "3" + 4);`
- (D) `System.out.print(1 + "234");`

31. Consider the following code segment, intended to find the position of an integer `targetValue` in `int[] a`.

```
int i = 0;
while (a[i] != targetValue)
{
 i++;
}
int position = i;
```

When will this code work as intended?

- (A) Always
  - (B) Only when `targetValue == a[0]`
  - (C) Only when `0 <= targetValue < a.length`
  - (D) Only when `targetValue` equals `a[i]` for some `i` such that `0 <= i < a.length`
32. What is printed when the following code segment is executed?

```
ArrayList<Integer> list = new ArrayList<Integer>();

for (int k = 1; k <= 6; k++)
{
 list.add(k);
}

for (int k = 0; k < 3; k++)
{
 Integer i = list.remove(k);
 list.add(i);
}

for (Integer i : list)
{
 System.out.print(i);
}
```

- (A) 123456
- (B) 246135
- (C) 456123
- (D) `IndexOutOfBoundsException`

33. Consider the following class.

```
public class Monster
{
 private String food;

 public void feed(String number)
 {
 if (food == null || food.compareTo(number) < 0)
 {
 food = number;
 }
 }

 public String get()
 {
 return food;
 }
}
```

What is printed when the following code in Monster's client class is executed?

```
Monster bfg = new Monster();
String[] numbers = {"One", "Two", "Three", "Four", "Five",
 "Six", "Seven", "Eight", "Nine", "Ten"};
for (String number : numbers)
{
 bfg.feed(number);
}
System.out.println(bfg.get());
```

- (A) One
- (B) Two
- (C) Ten
- (D) NullPointerException

34. What is the value of `count` after the following code segment is executed?

```
int count = 0;
for (int j = 1; j <= 2; j++)
{
 count++;
 for (int k = 1; k <= 3; k++)
 {
 count *= 2;
 }
}
```

- (A) 3  
(B) 8  
(C) 17  
(D) 72
35. Suppose a `Scanner` object `scan` is associated with a text file that is open for reading. The file contains three lines (with a blank second line), as follows:

|          |
|----------|
| I am Sam |
|----------|

|          |
|----------|
| Sam I am |
|----------|

Consider the following code segment.

```
int count = 0;
while (scan.hasNext())
{
 scan.nextLine();
 count++;
}
```

What is the value of `count` after this code runs, and what would its value be if we were to replace `nextLine` with `next`?

|     | nextLine | next |
|-----|----------|------|
|     | -----    |      |
| (A) | 1        | 3    |
| (B) | 2        | 3    |
| (C) | 2        | 6    |
| (D) | 3        | 6    |

36. Suppose an array `arr` contains 127 different values arranged in ascending order, and the most efficient searching algorithm is used to find a target value. How many elements of the array will be examined when the target equals `arr[39]`?

(A) 4  
(B) 5  
(C) 7  
(D) 63

37. Consider the following code segment.

```
int[] arr = {1, 2, 3, 4, 5, 6, 7, 8};

for (int k = 1; k <= 6; k += 2)
{
 arr[7] = arr[k];
 arr[k] = arr[k+1];
 arr[k+1] = arr[7];
}
```

Which of the following represents the contents of `arr` after the code segment has been executed?

(A) 1 3 2 5 4 7 6 6  
(B) 1 3 2 5 4 7 6 8  
(C) 2 1 4 3 6 5 8 7  
(D) 2 1 4 3 6 5 7 8

38. Which of the following code segments correctly traverses row-by-row a rectangular two-dimensional `int` array `m`?

(A)

```
for (int x : m)
{
 ...
}
```

(B)

```
for (int r : m)
{
 for (int c = 0; c < m.length; c++)
 {
 int x = m[r][c];
 ...
 }
}
```

(C)

```
for (int c = 0; c < m.length; c++)
{
 for (int r = 0; r < m[c].length; r++)
 {
 int x = m[r][c];
 ...
 }
}
```

(D)

```
for (int[] r : m)
{
 for (int c = 0; c < m[0].length; c++)
 {
 int x = r[c];
 ...
 }
}
```

39. Consider the following method with missing code.

```
/**
 * Given a text file and a string target as parameters,
 * prints all lines in the file that contain target.
 * Scanner sc is associated with the text file.
 * String target is to be detected in lines from sc.
 * Precondition: Scanner sc is associated with a text file
 * open for reading.
 */
public void grep(Scanner sc, String target)
{
 while (sc.hasNext())
 {
 < missing code >
 }
}
```

Which of the following can replace < *missing code* > so that the method works as specified?

- (A)
- ```
if (sc.nextLine().indexOf(target) >= 0)
{
    System.out.println(sc.nextLine());
}
```
- (B)
- ```
String str = sc.nextLine();
if (str.equals(target))
{
 System.out.println(str);
}
```
- (C)
- ```
String line = sc.nextLine();
if (line.indexOf(target) >= 0)
{
    System.out.print(line);
}
```
- (D)
- ```
String line = sc.nextLine();
if (line.indexOf(target) >= 0)
{
 System.out.println(line);
}
```

40. Consider the following code segment.

```
int[][] t = new int[2][3];

for (int i = 0; i < t.length; i++)
{
 for (int j = 0; j < t[0].length; j++)
 {
 t[i][j] = i + j + 1;
 }
}
```

What is the result when the code segment is executed?

(A) t holds the values

```
1 2 3
4 5 6
```

(B) t holds the values

```
1 2 0
2 3 0
```

(C) t holds the values

```
1 2 3
2 3 4
```

(D) t holds the values

```
3 4 5
4 5 6
```

41. Consider the following method.

```
/**
 * Returns the number of zeros in s.
 * Preconditions: s.length() = 31;
 * s consists of several 0's followed by several 1's
 * (s can also be all 0's or all 1's)
 */
public int countZeros(String s)
{
 int i = 0, j = 30;
 while (i <= j)
 {
 int k = (i + j) / 2;
 if (s.substring(k, k+1).equals("0"))
 {
 i = k + 1;
 }
 else
 {
 j = k - 1;
 }
 }
 return i;
}
```

How many iterations through the `while` loop will be made in the best and the worst case?

|     | Best case | Worst case |
|-----|-----------|------------|
| (A) | 1         | 5          |
| (B) | 4         | 5          |
| (C) | 5         | 5          |
| (D) | 4         | 15         |

42. Consider the following class `Test` with an incomplete `main` method.

```
public class Test
{
 private static int year = 2026;

 public Test(int yr) { year = yr; }

 public void increment() { year++; }

 public String toString() { return year + ""; }

 public static void main(String[] args)
 {
 < missing statements >
 }
}
```

Which of the following replacements of `< missing statements >` will result in a syntax error or print something other than 2026 when `main` is executed?

- (A) `System.out.println(this);`
- (B) `Test t = new Test(2026);`  
`System.out.println(t.toString());`
- (C) `Test t = new Test(26);`  
`System.out.println("20" + t);`
- (D) `Test t = new Test(2025);`  
`t.increment();`  
`System.out.println(t);`



# Practice Exam #1

## SECTION II

Time — 90 minutes  
Number of questions — 4  
Percent of total grade — 45

1. A college dorm administrator is using the Java class `Dorm` to track room assignments in the dorm. You will write a constructor and one method of this class.

A partial definition of the `Dorm` class is shown below.

```
public class Dorm
{
 /** The map of the dorm */
 private final String dormMap;

 /** The number of rooms in the dorm */
 private int numRooms;

 /**
 * Sets dormMap to map, computes the number
 * of rooms in the dorm, and sets numRooms
 * to that number.
 * Precondition: map is not an empty string
 */
 public Dorm(String map)
 { /* to be implemented in Part (b) */ }

 /**
 * Returns the total number of beds in the room whose
 * number is specified by the parameter room.
 * Precondition: 1 <= room <= numRooms
 */
 public int totalBeds(int room)
 { /* implementation not shown */ }

 /**
 * Returns the number of vacant beds in the room.
 * Precondition: 1 <= room <= numRooms
 */
 public int vacantBeds(int room)
 { /* implementation not shown */ }
```

*Continued*



```

/**
 * Returns the ratio of the number of assigned
 * beds to the total number of beds, expressed
 * as the percentage rounded to the nearest integer.
 * Rooms with one bed are skipped -- reserved for
 * dorm resident assistants.
 */
public int assignedPercentage()
{ /* to be implemented in Part (a) */ }

/* other instance variables and methods not shown */
}

```

- (a) Write the method `assignedPercentage` that computes and returns the ratio of assigned beds to total beds in the dorm, expressed as the percentage rounded to the nearest integer. The rooms with one bed are skipped — they are reserved for RAs (Resident Assistants). For example, if the total number of beds is 23, with one single reserved for an RA, and the number of assigned beds is 12 (excluding the RA's bed), `assignedPercentage` returns 55, which is  $100 * 12/22$ , rounded to the nearest integer.

Complete the `assignedPercentage` method below.

```

/**
 * Returns the ratio of the number of assigned
 * beds to the total number of beds, expressed
 * as the percentage rounded to the nearest integer.
 * Rooms with one bed are skipped -- reserved for
 * dorm resident assistants.
 */
public int assignedPercentage()

```

- (b) Write the constructor of the `Dorm` class. It takes a “map” of the dorm as a parameter and assigns it to the instance variable `dormMap`. It then computes the number of rooms in the dorm and saves that number in the instance variable `numRooms`.

The “map” of the dorm is represented by a string in the format shown below. In each room, an “x” character indicates an assigned bed and a “.” character represents a vacant bed. The rooms are separated by a single “/” character. Also, the “map” string starts and ends with a “/” character.

For example:

```
/xx.x/.xx/x./x..xx/x.x./x/.x../
```

This map indicates that the dorm has seven rooms. One “single” room is reserved for an RA. The percentage is calculated based on 22 beds, of which 12 are assigned and the rest vacant.

In writing your solution for Part (b), you may **not** use any arrays or `ArrayList`, and you may use only the `String` methods listed in the *Java Quick Reference*.

Complete the `Dorm` constructor below.

```
/**
 * Sets dormMap to map, computes the number
 * of rooms in the dorm, and sets numRooms
 * to that number.
 * Precondition: map is not an empty string
 */
public Dorm(String map)
```



Class information for this question

```
public class Dorm

private final String dormMap
private int numRooms

public Dorm(String map)
public int totalBeds(int room)
public int vacantBeds(int room)
public int assignedPercentage()
```

¶ For additional practice, implement the `totalBeds` and `vacantBeds` methods of the `Dorm` class. This is easy to do by using `String`'s `split` method and, perhaps, `replaceAll` for `vacantBeds`.

¶ Write a main method in another class to test your `Dorm` class.

2. The `ReviewedProduct` class, which you will write, tracks reviews for a product. A `ReviewedProduct` object is created by calling a constructor that takes one parameter, the product's name (a `String`).

The `ReviewedProduct` class contains an `addReview` method, which takes one integer parameter, the number of stars. The number of stars passed to `addReview` is an integer in the range from 1 to 5 — this can be assumed to be a precondition. `addReview` updates the relevant instance variables and returns a message (a `String`). If a product has been reviewed at least three times and more than half of the reviews are one-star reviews, the returned message should be “[*product name*] has been reviewed [*number*] times; too many one-star reviews”.

The following table contains a sample code execution sequence and the corresponding returns from the `addReview` method. The code execution sequence appears in a class other than `ReviewedProduct`.

| Statement                                                         | Return Value<br>(blank if no value)                           | Explanation                                                                                                                                     |
|-------------------------------------------------------------------|---------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>ReviewedProduct pr1 = new ReviewedProduct("Glog");</code>   |                                                               | <code>ReviewedProduct pr1</code> is created.                                                                                                    |
| <code>pr1.addReview(2);</code>                                    | "Glog has been reviewed only once"                            | A two-star review is added.                                                                                                                     |
| <code>pr1.addReview(5);</code>                                    | "Glog has been reviewed 2 times; average rating 3.5 stars"    | A five-star review is added. The average is $(2 + 5)/2$ rounded to the nearest half-star.                                                       |
| <code>pr1.addReview(4);</code>                                    | "Glog has been reviewed 3 times; average rating 3.5 stars"    | The average is now $(2 + 5 + 4)/3$ rounded to the nearest half-star.                                                                            |
| <code>pr1.addReview(1);</code>                                    | "Glog has been reviewed 4 times; average rating 3.0 stars"    | $(2 + 5 + 4 + 1)/4 = 3$                                                                                                                         |
| <code>ReviewedProduct pr2 = new ReviewedProduct("Widget");</code> |                                                               | <code>ReviewedProduct pr2</code> is created.                                                                                                    |
| <code>pr2.addReview(1);</code>                                    | "Widget has been reviewed only once"                          | A one-star review is added.                                                                                                                     |
| <code>pr2.addReview(3);</code>                                    | "Widget has been reviewed 2 times; average rating 2.0 stars"  | $(1 + 3)/2 = 2$                                                                                                                                 |
| <code>pr2.addReview(4);</code>                                    | "Widget has been reviewed 3 times; average rating 2.5 stars"  | The average of all reviews for <code>pr2</code> so far, rounded to the nearest half-star.                                                       |
| <code>pr2.addReview(1);</code>                                    | "Widget has been reviewed 4 times; average rating 2.5 stars"  | A one-star review is added.                                                                                                                     |
| <code>pr2.addReview(1);</code>                                    | "Widget has been reviewed 5 times; too many one-star reviews" | A one-star review is added. The total number of reviews is 5, which is $\geq 3$ ; 3 out of 5 reviews, more than half, are now one-star reviews. |

Write the complete `ReviewedProduct` class, including any required instance variables, the constructor, and the `addReview` method. Do not use any arrays or lists in your solution. Your implementation must meet the above specifications and conform to the examples shown in the table.

For additional practice, write an overloaded version of `addReview` that takes the second parameter, a brief review (a `String`). The method should add the review to the returned string, separated from the rest by a newline character.

For example, the output from

```
ReviewedProduct pr1 = new ReviewedProduct("Glog");
System.out.println(pr1.addReview(2) + "\n");
System.out.println(pr1.addReview(
 5, "What a great product!") + "\n");
System.out.println(pr1.addReview(
 2, "Didn't like it") + "\n");
System.out.println(pr1.addReview(3) + "\n");
```

should be

Glog reviewed only once

Glog has been reviewed 2 times; average rating 3.5 stars  
Last review 5 stars  
What a great product!

Glog has been reviewed 3 times; average rating 3.0 stars  
Last review 2 stars  
Didn't like it

Glog has been reviewed 4 times; average rating 3.0 stars

3. Memory Lane is a one-dimensional version of the popular card-matching game Memory. Several cards are arranged face down in a row on the table. Each card contains a word or a picture, and there are two identical cards for each word or picture (so the number of cards is even). A player opens any two cards; if they have the same word or picture on them, the player takes both cards. If the cards are different, the player puts them back, face down.

In the computer model of the game, a card is represented by an instance of the class `Card`. The `Card` class has a constructor that takes a `String` as a parameter and stores that string in an instance variable. The `Card` class also defines an `equals` method, which compares this card to another card based on the values of the strings they hold. A partial definition of the `Card` class is shown below.

```
public class Card
{
 public Card(String w)
 { /* implementation not shown */ }

 /**
 * Returns true if this card is equal to other;
 * otherwise returns false.
 */
 public boolean equals(Object other)
 { /* implementation not shown */ }

 /* instance variables and other methods not shown */
}
```

The class `MemoryLane` supplies several methods for playing the game. A partial implementation of this class is shown below.

```
public class MemoryLane
{
 /** Cards arranged in a row on the table */
 private ArrayList<Card> cards;

 /**
 * Constructs a list of cards in which each card
 * occurs exactly twice. The cards are arranged in
 * random order. words is an array of words from
 * which the cards are generated.
 * Precondition: words.length > 0
 */
 public MemoryLane(String[] words)
 { /* implementation not shown */ }

 /**
 * First checks whether the cards list is a valid
 * arrangement, that is, for each card in the list
 * there is exactly one other card equal to the
 * first one. Then compares the cards at indices k1 and k2;
 * if they are equal, removes these two cards
 * and returns true; otherwise leaves the cards list
 * unchanged and returns false.
 * Precondition: 0 <= k1 < k2 < cards.size()
 */
 public boolean openTwoCards(int k1, int k2)
 { /* to be implemented */ }

 /* other instance variables and methods not shown */
}
```

Write a boolean method `openTwoCards`. It first checks whether the cards list represents a valid configuration for the game — that is, for each card there is exactly one other card equal to it — and returns `false` if the arrangement is not valid. Then it compares the cards at the given two indices. If the cards are equal, then both are removed from `cards`, and `openTwoCards` returns `true`; otherwise, the `cards` list remains unchanged, and the method returns `false`.

Complete the method `openTwoCards` below.

```
/**
 * First checks whether the cards list is a valid
 * arrangement, that is, for each card in the list
 * there is exactly one other card equal to the
 * first one. Then compares the cards at indices k1 and k2;
 * if they are equal, removes these two cards
 * and returns true; otherwise leaves the cards list
 * unchanged and returns false.
 * Precondition: 0 <= k1 < k2 < cards.size()
 */
public boolean openTwoCards(int k1, int k2)
```

❖ ❖ ❖

Class information for this question

```
public class MemoryLane
 private ArrayList<Card> cards
 public MemoryLane(String[] words)
 public boolean openTwoCards(int k1, int k2)
```

 For additional practice, write the `Card` class. In addition to the constructor and the `equals` method explained in the introduction to this question, supply a `toString` method that returns the word on the card.

Also write and test a constructor for the `MemoryLane` class:

```
public MemoryLane(String[] words)
```

The constructor should initialize the `ArrayList cards` by producing a valid configuration of randomly ordered pairs of cards. The words on the cards come from the array `words`; two cards are created for each word in `words`, so the size of the list `cards` will be `2*words.length`.

Write a helper method `shuffle` that arranges the elements of `cards` in random order in such a way that each card has an equal probability of ending up in any location. Make this method `static`, reusable for any list of objects, as follows:

 `public static void shuffle(ArrayList items)`

4. In a traffic modeling application, a segment of a highway is represented as a 2D array of integers, in which each row represents one lane and each column represents a multilane cross-section at a given position. A value of 1 signifies a car is present, and a value of 0 signifies an empty slot. All the cars move in the same direction. For example, the stretch of highway below has 4 lanes.

```

00000000
00010010
01001000
00100000

```

  
 Direction of traffic

The model is implemented with the help of the class `Highway`. A partial definition of this class is shown below. You will write one method of this class.

```

public class Highway
{
 /**
 * hwy[row][col] = 1 if there is a car at position (row, col);
 * hwy[row][col] = 0 if there is no car at position (row, col)
 */
 private int[][] hwy;

 /** The probability of placing a car into a slot. */
 private double probability;

 /**
 * Constructs an empty Highway with a given number of
 * lanes and a given length.
 */
 public Highway(int lanes, int length, double p)
 { /* implementation not shown */ }

 /**
 * Moves all cars in all lanes forward by one position.
 * For each lane, the car in the last (rightmost) position
 * is removed, if present. Also, for each lane, a car is
 * added at the first position (column 0) in the same lane
 * with the probability saved in the instance variable
 * probability.
 */
 public void moveAllForward()
 { /* to be implemented */ }

 /* other instance variables and methods not shown */
}

```

Write the method `moveAllForward` of the `Highway` class. In this simplified version of the model, each car moves forward by one position. A car in the rightmost position (rightmost column) is removed. A new car is added in the leftmost position (column 0) randomly with the probability saved in the instance variable `probability`. You may not use any temporary arrays or lists.

Complete the method `moveAllForward` below.

```
/**
 * Moves all cars in all lanes forward by one position.
 * For each lane, the car in the last (rightmost) position
 * is removed, if present. Also, for each lane, a car is
 * added at the first position (column 0) in the same lane
 * with the probability saved in the instance variable
 * probability.
 */
public void moveAllForward()
```

❖ ❖ ❖

Class information for this question

```
public class Highway
private int[][] hwy
private double probability

public Highway(int lanes, int length, double p)
public void moveAllForward()
```

For additional practice, write `Highway`'s constructor and a `toString` method that converts the `hwy` array into a string with appropriately placed newline characters.

Suppose cars can also make lateral moves ("switch lanes"). A car can switch to an adjacent lane if that lane is within `hwy` bounds and there are no cars at the same position in other lanes in the direction of the lateral move. Add a method

```
public boolean canSwitchLanes(int row, int col, int dir)
```

to the `Highway` class. It returns `true` if a lateral move is possible in the specified direction for the car at `(row, col)` position. The `dir` parameter is `-1` for the move up and `1` for the move down.